

SYSTEM EXPLAINER

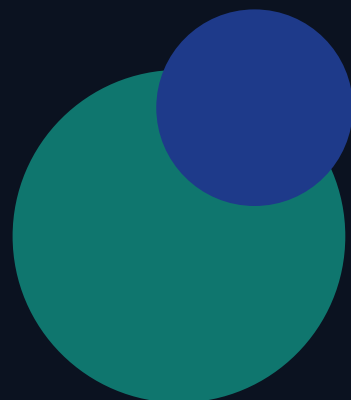
# The Codex App Factory

A plain-language explanation of the autonomous personal app-development system: what it does, how its AI team collaborates, where control lives, and why it remains safe and recoverable.

---

Codex App Factory | Keshav Peri | 19 August 2026

Subscription-first. App-agnostic. Human-approved production.



# The idea in one page

The Codex App Factory is a reusable way to turn a reviewed feature request into a tested draft pull request. It gives one person a small AI product team while keeping product intent, cost, privacy, and production control with the human owner.

## The promise

You describe the app and approve its rules. The factory can take one ready feature, clarify its scope, build it, test it, repair ordinary failures, and prepare evidence for review.

### Autonomous

It performs the repeatable engineering loop without needing step-by-step prompting.

### Recoverable

GitHub issues, branches, comments, CI, and pull requests preserve state between chats.

### Human-governed

It stops before production merges, billing, secrets, destructive changes, or major product decisions.

## What it is not

- It is not a single endlessly running chat that must remember everything.
- It is not a promise of unlimited usage; role agents consume additional tokens.
- It is not tied to football, web apps, npm, GitHub Pages, Supabase, or Vercel.
- It is not allowed to merge or publish production without Keshav's approval.

The private factory repository is **KeshavPeri/codex-app-factory**. Each real application gets its own independent repository and its own copy of the harness.

# The four-person product team

Codex is the platform; the factory assigns narrow responsibilities so the same agent is not simultaneously inventing scope, writing code, judging its own work, and controlling release state.

| Role            | Job                                                                                                   | Can edit code?      | Owns workflow state? |
|-----------------|-------------------------------------------------------------------------------------------------------|---------------------|----------------------|
| Orchestrator    | Chooses the ticket, delegates, tracks retries, manages Git and GitHub, and prepares the final review. | Coordinates only    | Yes                  |
| Product Manager | Checks scope, dependencies, acceptance criteria, and unresolved owner decisions.                      | No                  | No                   |
| Builder         | Implements one approved ticket and runs the project's real verification commands.                     | Yes - single writer | No                   |
| QA              | Independently reviews behavior, tests, regressions, and acceptance criteria.                          | No                  | No                   |

## Why one writer?

Multiple agents can explore or review in parallel, but simultaneous code writers in one working tree create conflicts and coordination overhead. The factory therefore uses one Builder and independent read-only gates.

## Codex-native building blocks

- AGENTS.md supplies durable repository rules at the start of work.
- Custom agent TOML files define the Product Manager, Builder, and QA roles.
- The \$codex-app-factory skill defines the repeatable orchestration algorithm.
- Scheduled tasks can launch the same workflow against a local project or isolated worktree.

# State that survives the chat

A factory run may stop, time out, hit a usage limit, or need an owner decision.

Recovery works because the important facts live in ordinary development artifacts, not hidden conversation memory.

| Artifact       | What it remembers                                                                        |
|----------------|------------------------------------------------------------------------------------------|
| Product brief  | What the app is for, its rules, data boundaries, non-goals, and definition of V1 done.   |
| Feature issue  | The current outcome, scope, exclusions, dependencies, and observable definition of done. |
| Workflow label | Whether the ticket is ready, building, waiting for review, or blocked.                   |
| Feature branch | Partial or completed implementation that a later run can recover.                        |
| Pull request   | The change, discussion, QA evidence, CI history, and owner decision.                     |
| Decisions log  | Expensive-to-reverse choices and the reasons they were made.                             |

## The issue state machine

| Label            | Meaning                      | Leaves this state when...              |
|------------------|------------------------------|----------------------------------------|
| factory:ready    | Reviewed and available       | A run claims it                        |
| factory:building | Active or recoverable work   | QA passes or the run blocks            |
| factory:review   | Draft PR awaits Keshav       | Keshav merges or requests revision     |
| factory:blocked  | One owner decision is needed | Keshav answers and returns it to ready |

### Fresh-session test

A new Codex chat should be able to reconstruct the app, selected feature, branch, remaining problem, and next action by reading the repository and GitHub state.

# What one autonomous run does

The run is deliberately bounded. It processes at most one ready ticket and has a hard repair limit.

1

**Cheap queue check**

Find the oldest open issue labelled `factory:ready` before loading the full codebase. Stop with 'Nothing ready' when the queue is empty.

2

**Recover or claim**

Reuse an existing branch or pull request when safe; otherwise claim the issue and create `codex/ticket-N-slug` from current main.

3

**Product gate**

The Product Manager returns `READY` with checks or `BLOCKED` with one owner-facing question.

4

**Build**

One Builder implements only the accepted scope and runs the commands defined for the project.

5

**Independent QA**

QA reviews the diff, tests the acceptance criteria, and reports concrete failures without editing code.

6

**Focused repair**

The same Builder gets no more than two focused revisions. Repeated failure blocks the ticket and preserves the branch.

7

**Prepare review**

On `PASS`, the orchestrator commits, pushes, opens or updates a draft PR, attaches evidence, and labels the issue `factory:review`.

8

**Stop at the owner gate**

Keshav reviews the result and decides whether to merge. The autonomous run ends before production.

# Control, safety, and escalation

Autonomy is useful only when its boundary is obvious. The factory distinguishes routine reversible engineering choices from actions that change authority, cost, privacy, or production state.

## Factory may decide

Focused implementation details, tests, refactors needed by the ticket, routine UI behavior, and other reversible choices inside approved scope.

## Factory must ask

Production merge, account creation, billing, API keys, secrets, destructive data operations, privacy exposure, or an unresolved high-impact product rule.

## Failure is a normal state

- A missing decision becomes one concise question on a blocked issue.
- A dead run leaves a branch that can be recovered instead of duplicated.
- A test failure receives a focused revision rather than a broad rewrite.
- Two failed revisions stop the loop; the ticket is split or re-specified by the owner.
- A missing preview is reported honestly and replaced with local QA evidence when appropriate.

## The invariant

The factory may prepare production-quality work. It may not silently widen its authority in order to finish.

# App-agnostic by design

The reusable layer describes how work moves, not what the product is or which technology it uses.

| Reusable factory layer                | Per-app layer                                      |
|---------------------------------------|----------------------------------------------------|
| Agent roles and permissions           | Product purpose, users, and domain rules           |
| Issue labels and branch convention    | Language, framework, and architecture              |
| One-ticket run and two-revision limit | Data model, privacy, and integrations              |
| Independent QA and CI requirement     | Actual test, lint, build, run, and deploy commands |
| Manual production gate                | Visual direction and device support                |

## Profiles

The generic template selects no framework, language, database, or host. Optional profiles overlay stack-specific files. The included static-web profile adds npm CI and GitHub Pages deployment; future profiles can cover mobile, desktop, backend services, data tools, or other stacks.

### Where app rules belong

Football scoring, workout privacy, finance calculations, household workflows, or any other domain logic belongs in that app's product brief and tests - never in the shared factory template.

# Manual and scheduled operation

The same skill powers both an interactive run and a scheduled run. The difference is only how the run starts.

## Manual

Open the app repository in Codex and ask: Use \$codex-app-factory to process the next ready feature. Best for setup, smoke tests, urgent work, and debugging.

## Scheduled

Create a Codex desktop scheduled task against the app repository or an isolated worktree using factory/AUTOMATION-PROMPT.md.

## Local automation constraint

A local scheduled task can work on local project files, but the computer must remain available and the Codex desktop app must be running. A sleeping laptop is not an always-on build server.

## Usage posture

- The queue is checked before expensive repository exploration.
- Each run processes only one feature.
- Agents receive narrow responsibilities and focused context.
- Only one Builder writes code.
- The run stops after two failed revisions instead of looping indefinitely.
- Subagents improve separation and review quality, but they consume more tokens than a comparable single-agent run.

## Current design

Subscription-first local Codex. No OPENAI\_API\_KEY is configured by the factory. An always-on API-backed runner would be a separate cost and architecture decision.

# Where everything lives

| Location                          | Purpose                                           |
|-----------------------------------|---------------------------------------------------|
| AGENTS.md                         | Durable repository rules and authority boundaries |
| .codex/agents/*.toml              | Product Manager, Builder, and QA definitions      |
| .agents/skills/codex-app-factory/ | The orchestration workflow Codex invokes          |
| docs/product-brief.md             | Product source of truth                           |
| docs/design-reference.md          | Visual and interaction direction                  |
| docs/decisions.md                 | Important decisions and reasons                   |
| factory/PROJECT-CONFIG.md         | Real setup, verification, QA, and deploy commands |
| factory/AUTOMATION-PROMPT.md      | Manual or scheduled factory entry prompt          |
| factory/REVIEW-PACKET.md          | Required pull-request evidence                    |
| GitHub issue / branch / PR        | Durable feature state and review history          |

## Current status

The private factory repository exists, and its skill, agent TOML, YAML, shell scripts, scaffolder, and Codex instruction loading have been validated. The first real app will be created only after its design workshop supplies the product brief and backlog.

## Independent from Claude

The Codex factory and future Codex apps live separately from /Users/keshav/Projects/app-factory and /Users/keshav/Projects/fpl-advisor. They do not share automations or workflow state.

# Official references and glossary

## Official Codex references

- AGENTS.md: <https://learn.chatgpt.com/docs/agent-configuration/agents-md>
- Subagents and custom agents: <https://learn.chatgpt.com/docs/agent-configuration/subagents>
- Skills: <https://learn.chatgpt.com/docs/build-skills>
- Scheduled tasks: <https://learn.chatgpt.com/docs/automations?surface=app>

## Glossary

| Term     | Meaning here                                                                         |
|----------|--------------------------------------------------------------------------------------|
| Factory  | The reusable instructions, roles, templates, and workflow that each app receives.    |
| Run      | One bounded attempt to process at most one ready feature.                            |
| Skill    | A reusable workflow Codex loads when the task matches or it is invoked by name.      |
| Subagent | A specialized delegated agent working under the primary orchestrator.                |
| CI       | Deterministic automated checks run by the repository, usually on every pull request. |
| Draft PR | A pull request prepared for review but not yet approved for production.              |
| Profile  | An optional set of stack-specific files layered onto the generic app template.       |

### The simplest summary

A feature queue, a careful product gate, one code writer, an independent QA gate, deterministic tests, recoverable Git state, and a human production decision.